PYTHON PROGRAMMING



21 Days Challenge

Day-12

Day-12

FUNCTION POWER-UPS - MASTERING LAMBDA, MAP & FILTER, RECURSION, CLOSURES, AND DECORATORS.



Instructor



Abdullah Al Zubaer

Instructor – Artificial Intelligence & Data Science

Gobeshona Learning Academy.

B.Sc. in Internet of Things and Robotics Engineering

Gazipur Digital University, Gazipur, Bangladesh

Table Of Content

- Function Recursion
- Lambda Function
- Map Function
- Filter Function
- Reduce Function
- Function Decorators
- Function Generators
- Function Closures

Function Recursion

A function calling itself until a base condition is met.

```
def factorial(n):
    if n == 1:
        return 1
    return n * factorial(n-1)

print(factorial(5))
```

Output:

```
120
```

Lambda Function

Anonymous function use for simple operations.

```
square = lambda x: x*x
print(square(5))
```

Output:

```
25
```

Map Function

Applies a function to each element in a list use for quick transformations.

```
nums = [1, 2, 3, 4]
squares = list(map(lambda x: x*x, nums))
print(squares)
```

Output:

```
[1, 4, 9, 16]
```

Filter Function

Selects items based on a conditionuse to filter out unwanted values.

```
nums = [1, 2, 3, 4, 5]
evens = list(filter(lambda x: x%2==0, nums))
print(evens)
```

Output:

```
[2, 4]
```

Reduce Function

Reduces a list into a single value use for cumulative operations.

```
from functools import reduce


nums = [1, 2, 3, 4]
total = reduce(lambda x, y: x+y, nums)
print(total)
```

Output:

```
10
```

Function Decorators

Functions that wrap other functions to add extra features.

```
def my_decorator(func):
    def wrapper():
        print("Before function runs")
        func()
        print("After function runs")
    return wrapper

@my_decorator
def say_hello():
    print("Hello!")
```

Output:

```
Before function runs
Hello!
After function runs
```

Function Generators

A special function that returns values one by one using yield instead of return.

```
def count_up_to(n):
    i = 1
    while i <= n:
        yield i
        i += 1


for num in count_up_to(5):
    print(num)
```

```
Output:
1
2
3
4
5
```

Function Closures

Closure is a function that remembers variables from the outer function, even after the outer function has finished executing.

```
def outer_func(msg):
    def inner_func():
        print("Message:", msg)
    return inner_func


my_func = outer_func("Hello World")
my_func()
```

Output:

```
Message: Hello World
```

Summary

- **Recursion** → A function calling itself to solve problems step by step.
- **Lambda** → Small, one-line anonymous function.
- **map()** → Applies a function to each element of a list.
- **filter()** → Keeps only elements that satisfy a condition.
- **reduce()** → Combines all elements into a single result.
- **Decorator** → Adds extra functionality to existing functions.
- **Generator** → Yields values one by one, memory efficient.
- **Closure** → Inner function that remembers outer function variables.

QUESTION SESSION

THANK YOU

https://gobeshona.com.bd/